

Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

Course: Parallel and Distributed Computing

Project Title: Parallel Log Analyzer for Large Server Logs

Group Members:

Muhammad Askee Iqbal - CS143
Azeem Mohamed Husain - CS142
Abdhur Rahman - CS141

Project Proposal

1. Abstract

In modern computing environments, servers generate vast amounts of log data that must be analyzed to monitor performance, detect errors, and ensure security. Analyzing these large log files sequentially is time-consuming and inefficient. This project proposes the development of a Parallel Log Analyzer that leverages Parallel and Distributed Computing concepts to process and analyze massive log files efficiently. The log file will be divided into multiple chunks and processed concurrently using parallel processes or threads. Each process will extract key information—such as the number of error, warning, and informational messages—and the results will be aggregated to produce a complete summary. The system will measure execution time and compute the performance gain compared to sequential execution, demonstrating the benefits of parallel processing for real-world data analysis tasks.

2. Objectives

- To design and implement a system that can analyze large server log files in parallel.
- To demonstrate the use of data parallelism and task synchronization in a practical application.
- To measure and compare the execution time of serial vs. parallel log analysis.

- To visualize results (e.g., error counts, processing time, and speedup) for better understanding.
- To enhance understanding of how parallel computing improves performance in data-heavy applications.

3. Methodology

1. Input Data Preparation: A large text file (e.g., simulated web server log) will be used as input. Each log entry contains timestamps, event levels (INFO, WARNING, ERROR), and messages.
2. Data Partitioning: The log file will be divided into equal-sized chunks and each chunk will be assigned to a separate processor or thread.
3. Parallel Processing: Each worker thread/process will count occurrences of key log types (INFO, WARNING, ERROR). Intermediate results will be stored temporarily.
4. Result Aggregation: The main process will combine all partial results into a final summary, computing overall execution time and performance metrics.
5. Performance Comparison: The program will run in both serial and parallel modes. Speedup and efficiency will be measured.
6. Visualization (Optional): Results and performance improvements will be displayed using simple graphs or charts.

4. Tools and Technologies

- Programming Language: Python 3
- Libraries: multiprocessing, time, matplotlib
- Platform: Windows or Linux
- Dataset: Large synthetic or real server log file
- (Optional): OpenMP (C/C++) for advanced implementation

5. Expected Output

A working system capable of analyzing large log files in parallel.
Summary report displaying counts of INFO, WARNING, and ERROR messages.
Comparison of serial vs. parallel execution times and corresponding speedup ratio.
Graphical visualization of analysis results and performance metrics.

Example Output:

Total Lines Processed: 1,000,000
INFO: 8420
WARNING: 1200
ERROR: 400
Serial Time: 12.8s
Parallel Time: 3.4s

Speedup: 3.76x

6. Expected Outcome and Learning

This project will help students understand how data decomposition, process synchronization, and performance analysis are applied in real-world parallel systems. It also illustrates how parallelism can drastically improve processing efficiency for data-intensive applications like log analysis, which are common in enterprise environments.

7. References

- Ananth Grama et al., Introduction to Parallel Computing, 2nd Edition, Addison-Wesley.
- Official Python Documentation: <https://docs.python.org/3/library/multiprocessing.html>
- Foster, Ian. Designing and Building Parallel Programs, Addison-Wesley, 1995.